

Relatório Técnico

PROJETO EVENTMASTER

Grupo:

Alice Postai Nunes

Débora D'Angelo Reina de Araujo

Fabício Sanches

Introdução

O projeto EventMaster foi desenvolvido com o objetivo de substituir um sistema monolítico de venda de ingressos que apresentava diversos problemas operacionais durante períodos de alta demanda. Em grandes lançamentos e pré-vendas, o sistema sofria com quedas frequentes, lentidão, inconsistência de dados e dificuldades de escalabilidade. Além disso, a arquitetura anterior possuía forte acoplamento entre módulos, dificultando manutenção, evolução e aplicação de mecanismos modernos de segurança.

Para resolver esses problemas, foi proposta uma nova arquitetura baseada em microsserviços, utilizando conceitos de Clean Architecture, Domain-Driven Design (DDD), comunicação assíncrona com Kafka e autenticação baseada em JWT. O principal objetivo da solução é garantir alta disponibilidade, escalabilidade horizontal, isolamento de responsabilidades e maior resiliência em cenários de pico de acesso.

Arquitetura

A nova arquitetura foi construída utilizando o modelo de microsserviços, onde cada serviço possui responsabilidade isolada e banco de dados próprio. Essa abordagem reduz o acoplamento entre módulos e permite que cada componente seja escalado independentemente conforme a necessidade do sistema.

O fluxo principal da aplicação começa nas aplicações (web e mobile), que se comunicam inicialmente com o API Gateway. O Gateway atua como ponto central de entrada da plataforma, sendo responsável por roteamento, autenticação, validação de JWT, observabilidade e controle de acesso.

A solução foi dividida nos seguintes serviços:

- User Service: Gerenciamento de autenticação e usuários;
- Event Service: Gerenciamento de eventos
- Ticket Service: Gerenciamento de ingressos
- Order Service: Gerenciamento de pedidos
- Payment Service: Simula o serviço de pagamento
- API Gateway: Ponto único de acesso.

Cada serviço possui banco de dados isolado utilizando MySQL, seguindo o padrão banco por serviço, isso garante independência entre os domínios e evita impactos no sistema inteiro em caso de falha em um dos serviços ou do banco de dados.

A comunicação entre os serviços ocorre de forma síncrona e assíncrona. Para fluxos críticos de negócio e eventos de domínio, foi utilizado Apache Kafka como barramento de eventos da arquitetura.

Modelagem do Sistema

A modelagem do sistema foi baseada em Domain-Driven Design para garantir melhor separação de responsabilidades e maior alinhamento com as regras de negócio.

Foram identificados cinco principais bounded contexts:

- Usuário e Autenticação
- Catálogo de Evento
- Ingresso
- Pedido
- Pagamento

Contexto de Catálogo de Evento

Responsável pelo gerenciamento de funcionalidades relacionadas à criação, consulta e manutenção de eventos, leitura de informações públicas, busca e navegação da plataforma.

Como esse domínio possui alta quantidade de leitura, foi utilizado Redis como camada de cache para reduzir acesso ao banco de dados e melhorar performance.

Contexto de Vendas

Responsável pelos fluxos transacionais da aplicação, incluindo pedidos, reserva de ingressos, pagamentos e confirmação de compra. Esse contexto concentra regras críticas de consistência e utiliza comunicação assíncrona via Kafka para garantir desacoplamento e resiliência.

Dentro desse contexto foram criados os microserviços de:

- Ticket Service – Gerencia de Ingressos
- Order Service – Gerencia de Pedidos
- Payment Service – Simula o pagamento

Processamento Assíncrono e Escalabilidade

Um dos principais problemas do sistema monolítico anterior era o acoplamento excessivo entre operações críticas. Para resolver isso, foi adotada uma arquitetura orientada a eventos utilizando Apache Kafka.

O Kafka atua como intermediador entre os serviços, permitindo que eventos sejam publicados e consumidos sem dependência direta entre aplicações. Isso reduz gargalos, melhora escalabilidade e evita falhas em cascata.

Os principais eventos utilizados são:

- **EVENTO_CRIADO** – Postado na fila quando um novo evento é criado, sabendo-se então quantos ingressos poderão ser emitidos. O serviço de ingressos consome esse evento e faz a carga de 95% da capacidade cadastrada no Redis, com status Disponível;
- **PEDIDO_REALIZADO** – Postado na fila quando o pedido é feito, esse evento é escutado pelo serviço de ingressos e então é feita a reserva da quantidade de ingressos do pedido.
- **PAGAMENTO_CONFIRMADO** - Postado na fila quando o serviço de pagamento processa, é escutado pelos serviços de ingresso que muda o status do ingresso para confirmado e o serviço de pedido que muda seu estado para confirmado.
- **PAGAMENTO_CANCELADO** – Postado na fila quando o serviço de pagamento tem algum problema, é escutado pelos serviços de ingresso que libera o ingresso para a venda futura e o serviço de pedido que muda seu estado para cancelado.

Padrão SAGA e Consistência Distribuída

Como cada microsserviço possui banco próprio, não é possível utilizar transações ACID tradicionais envolvendo múltiplos serviços. Para resolver esse problema, foi adotado o padrão SAGA.

O fluxo funciona da seguinte maneira:

- (1) O pedido é criado no Order Service;
- (2) O Ticket Service reserva o ingresso;
- (3) O Payment Service processa o pagamento;
- (4) Se o pagamento for aprovado, o ingresso é marcado como confirmado;
- (5) Caso o pagamento falhe, um evento de compensação cancela a reserva do ingresso e muda o status do pedido para cancelado.

Redis e Estratégia Hot-Seat

Durante grandes pré-vendas, milhares de usuários podem tentar comprar o mesmo ingresso simultaneamente. Para evitar contenção excessiva no banco de dados, foi utilizada uma estratégia baseada em Redis.

O Redis mantém contadores em memória para controle de disponibilidade dos ingressos mais disputados. Operações atômicas de decremento permitem reservar ingressos com alta performance e baixa latência.

Posteriormente, as informações são persistidas no banco através de write-behind, reduzindo a quantidade de operações síncronas no banco relacional.

Essa estratégia melhora significativamente o throughput da aplicação em cenários de alta concorrência.

Segurança

O sistema utiliza autenticação baseada em JWT (JSON Web Token). O fluxo de autenticação funciona da seguinte maneira:

1. O usuário realiza login;
2. O User Service gera um JWT assinado;
3. O token é enviado ao cliente;
4. O API Gateway valida o token;
5. Os microsserviços também realizam validação própria do JWT.

Dessa forma, é garantido que um eventual bypass no Gateway não comprometa completamente a segurança interna.

O projeto utiliza Spring Security e controle de acesso baseado em roles:

- **ROLE_ADMIN;**
- **ROLE_USER;**

Além disso, foi implementado um mecanismo de blacklist de tokens revogados, permitindo logout seguro mesmo utilizando autenticação stateless.

Proteção Contra OWASP Top 10

A proteção contra Injection ocorre através do uso de JPA Repository e validação de entrada, evitando concatenação manual de SQL. O controle de acesso é realizado utilizando Spring Security com `@PreAuthorize`, impedindo acesso indevido a recursos protegidos. As senhas são protegidas utilizando BCrypt, tornando o armazenamento criptografado. Os JWTs possuem assinatura digital, expiração, validação de issuer e validação de role. Também foram implementados logs estruturados e correlation-id para rastreabilidade e auditoria.

Observabilidade e Monitoramento

A arquitetura implementa rastreamento distribuído utilizando correlation-id propagado entre os serviços. O Gateway possui filtros globais responsáveis por geração de correlation-id, medição de latência, logging de requisições e rastreamento de falhas. Além disso, a solução prevê integração com Grafana para monitoramento em tempo real do fluxo de vendas e comportamento do sistema.

Os relatórios financeiros e consolidações diárias são executados em processamento batch, enquanto métricas operacionais utilizam processamento stream em tempo real.

Clean Architecture e SOLID

O projeto foi estruturado seguindo princípios de Clean Architecture, separando claramente os controllers, services, repositories, entidades de domínio, DTOs e camada de segurança.

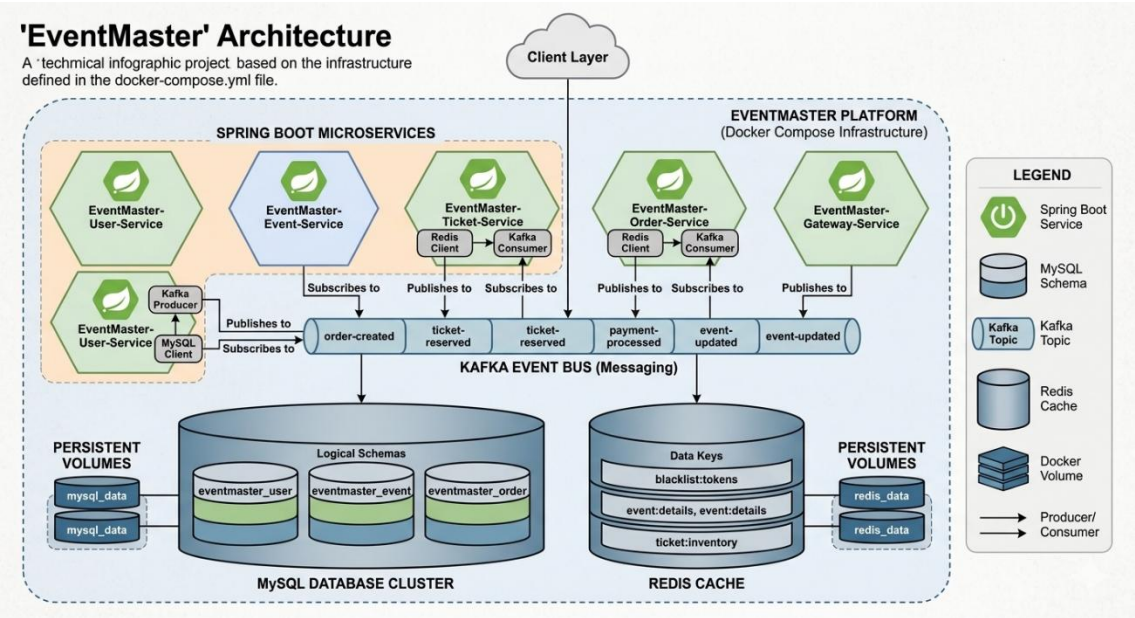
Essa organização facilita manutenção, testes e evolução futura do sistema.

Os princípios SOLID também foram aplicados. Um exemplo é o TokenService, responsável exclusivamente pelo gerenciamento de JWT, enquanto o TokenBlacklistService possui responsabilidade isolada para controle de revogação de tokens.

A injeção de dependências via Spring reduz o acoplamento e melhora a testabilidade.

Estrutura do Projeto

O projeto foi organizado como monorepo Maven, permitindo gerenciamento centralizado de dependências e build.



O gateway-service atua como camada de borda da aplicação, enquanto o user-service concentra autenticação e gerenciamento de usuários.

Tabela de URLs (Swagger & OpenAPI)

Aqui está a tabela completa com todos os endereços do **Swagger UI** e do **OpenAPI JSON** do projeto **EventMaster**:

Microsserviço	Porta Local	URL do Swagger UI	Rota Agregada via Gateway
gateway-service	8081	http://localhost:8081/swagger-ui.html	Interface Centralizadora Geral
user-service	8080	http://localhost:8080/swagger-ui.html	http://localhost:8081/api/users/swagger-ui/index.html
event-service	8082	http://localhost:8082/swagger-ui.html	http://localhost:8081/api/events/swagger-ui/index.html
ticket-service	8083	http://localhost:8083/swagger-ui.html	http://localhost:8081/api/tickets/swagger-ui/index.html
order-service	8084	http://localhost:8084/swagger-ui.html	http://localhost:8081/api/orders/swagger-ui/index.html
payment-service	8086	http://localhost:8086/swagger-ui.html	Não aplicável (Consumidor Kafka)

API do Gateway

Ao acessar o link principal do Gateway (<http://localhost:8081/swagger-ui.html>), um menu de seleção (*Select a definition*) estará disponível no canto superior direito. Ele permite alternar dinamicamente entre as documentações de todos os microserviços sem precisar ficar trocando de porta no navegador.

Nota sobre o Payment Service: Embora o payment-service gere seu JSON OpenAPI localmente na porta 8086, ele não possui rotas de API públicas expostas no Gateway, pois sua arquitetura no ecossistema é estritamente reativa (orientada a eventos assíncronos disparados via tópicos do Apache Kafka).

Estratégia de Testes

A solução segue o conceito de Pirâmide de Testes. Os testes unitários cobrem regras de negócio, autenticação, geração e validação de JWT e serviços internos.

Os testes de integração validam endpoints REST, segurança, persistência e integração entre camadas.

Também foram previstos testes BDD para cenários completos de autenticação e autorização, utilizando ferramentas como JUnit, Mockito, Spring Boot Test e Cucumber.

Testes Autenticados

Para testar os endpoints protegidos diretamente por essas URLs, lembre-se de realizar o login no user-service (POST /auth/login), copiar o token gerado, clicar no botão **Authorize** do Swagger e inseri-lo no padrão Bearer <seu_token>.

As credenciais para login são:

Usuário: eventmaster@teste.com

Senha: event@123

Preparação de Ambiente

Para obter o projeto clone o seguinte repositório:

<https://github.com/fsanchesbr001/eventmaster-mod1-ada>

Por tratar-se de aplicação que utiliza containerização, para ter o ambiente funcional execute o comando:

.\subir-plataforma.ps1 -Mode Compose no powershell.

Conclusão

A arquitetura proposta para o EventMaster resolve os principais problemas do antigo sistema monolítico, fornecendo uma solução mais escalável e resiliente. A utilização de microsserviços, Kafka, Redis, JWT, DDD e Clean Architecture permite que a plataforma suporte grandes volumes de acesso simultâneo com maior disponibilidade e segurança. Além disso, a separação de domínios e o desacoplamento entre serviços tornam o sistema mais simples de evoluir e manter ao longo do tempo, garantindo uma base sólida para futuras possíveis expansões da plataforma.